Design and Development of a Medical Equipment Asset Management and Decision-Support System for Hospital Level Implementation in Ethiopia

By:

ABEL ZEREABRUK	UGR/2672/14
ABIGIYA CHINA	UGR/9769/14
BEAMLAK FEKADU	UGR/2356/14
ELDANA ABIY	UGR/6935/14
FITSUM FESEHA	UGR/4940/14
YODIT ALMAW	UGR/6368/14

BSc. Thesis Progress Report

Submitted to the School of Biomedical Engineering

In Partial Fulfillment of Requirement of the Degree of Bachelor of Science

(Biomedical Engineering)

Advisor:

Wubshet Shimels

ADDIS ABABA UNIVERSITY

COLLEGE OF TECHNOLOGY AND BUILT ENVIRONMENT

SCHOOL OF BIOMEDICAL ENGINEERING

ADDIS ABABA, ETHIOPIA

March, 2026

Signature and Approval Page

1. Abel Zereabruk	UGR/2672/14	_____
Name	ID. No.	Signature
2. Abigiya China	UGR/9769/14	_____
Name	ID. No.	Signature
3. Beamlak Fekadu	UGR/2356/14	_____
Name	ID. No.	Signature
4. Eldana Abiy	UGR/6935/14	_____
Name	ID. No.	Signature
5. Fitsum Feseha	UGR/4940/14	_____
Name	ID. No.	Signature
6. Yodit Almaw	UGR/6368/14	_____
Name	ID. No.	Signature

Signature of Advisors

Wubshet Shimels	_____	_____
Name	Signature	Date

Approved By

_____	_____	_____
Undergraduate Coordinator	Signature	Date
_____	_____	_____
Center Head	Signature	Date
_____	_____	_____
Undergraduate Director	Signature	Date

ABSTRACT

This progress report documents the current state of the Biomedical Equipment Resource Management System (BMERMS), a web-based platform for medical equipment asset management and decision support intended for hospital-level implementation in Ethiopia. The system addresses persistent gaps in equipment lifecycle visibility, maintenance coordination, and data-driven decision making in resource-constrained healthcare facilities. The implemented prototype is built on Next.js, TypeScript, and a Supabase/PostgreSQL backend, and includes operational modules for inventory and asset registry, maintenance request and work-order management, preventive maintenance, calibration, spare parts and logistics, procurement, training, and disposal. On top of these operational modules, an analytical layer computes Risk Priority Number (RPN), Mean Time Between Failures (MTBF), Mean Time To Repair (MTTR), Availability, Preventive Maintenance Compliance (PMC), composite performance scores, equipment health, clinical readiness, and a multi-criteria Replacement Priority Index (RPI). A dedicated Decision Support Center surfaces a triage queue, recommendation flags, and escalation events to assist biomedical engineering departments. The current version remains a prototype operating on a seeded demonstration dataset; validation against real hospital inventory and maintenance records, end-to-end role testing, and a hospital-level pilot remain as required next steps before final defense.

ACKNOWLEDGEMENTS

First, the project team would like to thank God the Almighty for strength, guidance, and good health throughout the duration of this work. The team expresses sincere gratitude to the project advisor, Mr. Wubshet Shimels, for his continued guidance, technical insight, and constructive feedback during the transition from proposal-stage design to a working prototype. The team also extends appreciation to the staff of the School of Biomedical Engineering at the College of Technology and Built Environment, Addis Ababa University, for their academic support, and to the biomedical engineering staff of Yekatit 12 Hospital for ongoing willingness to host this hospital-level prototype and to share insights into routine clinical engineering workflows.

Table of Contents

List of Tables

List of Figures

List of Abbreviations

API	Application Programming Interface
BME	Biomedical Engineering / Biomedical Engineer
BMERMS	Biomedical Engineering Resource Management System
CMMS	Computerized Maintenance Management System
DB	Database
DSS	Decision-Support System
FMEA	Failure Mode and Effects Analysis
ICU	Intensive Care Unit
KPI	Key Performance Indicator
MEMIS	Medical Equipment Management Information System
MTBF	Mean Time Between Failures
MTTR	Mean Time To Repair
Next.js	A React-based web application framework
PM	Preventive Maintenance
PMC	Preventive Maintenance Compliance
PostgreSQL	Open-source relational database management system
QR	Quick Response (machine-readable code)
RLS	Row-Level Security
RPI	Replacement Priority Index
RPN	Risk Priority Number
SQL	Structured Query Language
Supabase	Open-source backend-as-a-service platform built on PostgreSQL
UI	User Interface

CHAPTER ONE: INTRODUCTION

1.1 Background

Medical equipment is a critical resource in modern healthcare delivery, and the safe, reliable, and economically sustainable operation of that equipment depends on continuous monitoring, structured maintenance, and informed managerial decisions. In resource-constrained healthcare facilities, including many hospitals in Ethiopia, biomedical engineering departments often manage equipment lifecycles using fragmented paper records, generic spreadsheets, or isolated tools that do not capture the full chain of installation, operation, maintenance, calibration, spare-part logistics, and decommissioning. The result is limited visibility into equipment condition, delayed corrective action, missed preventive maintenance, and weak evidence to support replacement and procurement decisions.

This project, titled the Biomedical Equipment Resource Management System (BMERMS), targets these gaps by combining an integrated equipment lifecycle management platform with an explicit analytical and decision-support layer. The platform is intended for hospital-level implementation in Ethiopian hospitals and is designed for use by biomedical engineering departments, technicians, department users, and administrators in resource-constrained healthcare facilities.

1.2 Purpose of the Progress Report

This report serves three primary purposes. First, it formally records the implementation progress achieved between the proposal stage and the current reporting period. Second, it demonstrates how the implemented system aligns with the analytical and operational commitments described in the proposal, including the specific equations and decision-support outputs that the system was required to produce. Third, it identifies the limitations that remain, the prioritised activities to be completed before final defense, and the additional evaluation steps planned.

1.3 Overview of the Proposed System

The proposed system is a hospital-level Medical Equipment Asset Management and Decision-Support System that integrates equipment inventory management, maintenance and work-order tracking, preventive maintenance scheduling and completion, calibration management, spare parts and logistics, training, disposal, reporting, and analytics into a single unified web application. The

system intentionally concentrates on hospital-level operational and analytical decision support, rather than national-scale information flow.

Beyond conventional record-keeping, the platform was committed to producing analytical decision-support outputs including reliability metrics, risk-prioritisation scores derived from FMEA, preventive maintenance compliance percentages, replacement priority rankings, and equipment performance scores. These outputs were intended to convert routine maintenance and inventory data into ranked, explainable engineering priorities that would help biomedical engineering teams allocate limited maintenance time and resources more effectively.

1.4 Scope of the Current Implementation

The current scope is a hospital-level prototype implemented as a Next.js application backed by a Supabase/PostgreSQL database, with row-level security (RLS) policies, role-based access, and a system-wide AI copilot. The prototype operates on a seeded demonstration dataset and a structured set of database migrations. Validation against real hospital inventory and maintenance records, end-to-end role testing, and a hospital-level pilot in a selected Ethiopian hospital setting remain as required next steps. Where repository labels or seed data still reference earlier partner-hospital naming, those references are treated as codebase artifacts that should be generalized prior to final examiner submission and are not part of the formal scope of this report.

CHAPTER TWO: PROPOSED SYSTEM AND DEVELOPMENT CONTEXT

2.1 Summary of the Original Proposal

The original thesis proposal motivated the design of an integrated, multi-module medical equipment management platform that goes beyond a simple inventory registry. The proposed system was specified to support equipment registration, maintenance and work-order management, preventive maintenance, calibration, spare-part logistics, training, and disposal, while exposing analytical computations rooted in established reliability and risk indicators. Equations 1 through 7 of the proposal define the analytical foundations: Equation 1 defines RPN as the product of severity, occurrence, and detectability; Equations 2 through 4 define availability, MTBF, and MTTR; Equation 5 defines preventive maintenance compliance; Equation 6 specifies min-max normalization; and Equation 7 specifies weighted-sum aggregation for composite scoring.

2.2 Hospital-Level Implementation Context

The implementation context for BMERMS is a hospital-level setting in Ethiopia, characterized by mixed-criticality medical equipment fleets, limited technician capacity, intermittent connectivity, and constrained spare-part availability. Hospital biomedical engineering departments in such settings must balance reactive repair workload with preventive maintenance, calibration cycles, and lifecycle decisions about replacement and procurement. The system is therefore designed for resource-constrained healthcare facilities and is intended to support multiple user roles in parallel, including department users who originate fault reports, technicians who execute work, store users who manage spare parts, and administrators who oversee planning, escalation, and decision-making.

2.3 Relationship to MEMIS and MEMIS 2.0 Concepts

The Medical Equipment Management Information System (MEMIS) introduced by the Ethiopian Ministry of Health provides a structured national framework for equipment registration, requesting, reporting, logistics, settings, user management, and helpdesk-style workflows across the health system. The implemented BMERMS prototype is conceptually aligned with MEMIS in that the operational modules use the same lifecycle vocabulary and request categories. A central "Requests Hub" page mirrors the MEMIS 2.0 request taxonomy, exposing distinct entry points for

installation requests, procurement requests, specification requests, training requests, curative maintenance requests, calibration requests, and disposal requests.

However, the BMERMS prototype is not intended to replicate or replace national MEMIS infrastructure. Instead, it is a hospital-level extension that adds richer analytical and decision-support capabilities on top of the operational lifecycle. The repository contains a dedicated migration, identified internally as the MEMIS 2.0 decision-support and guardrails migration, which adds procurement tracking, equipment health snapshots, clinical readiness snapshots, triage queues, repeat repair flags, escalation rules and events, workload-capacity snapshots, inspection templates, and offline sync support. A second migration introduces the corresponding row-level security policies and the snapshot-refresh procedure. These additions reflect the project team's effort to preserve MEMIS-style operational alignment while extending the platform with hospital-level decision-support intelligence.

2.4 Technology Stack

The implementation uses a modern, layered full-stack architecture suitable for hospital deployment. Table 1 summarises the principal technologies used in each layer of the system as currently implemented in the repository.

Table 1. Technology Stack Used in the Current Implementation

Layer	Technology	Purpose in the System
Frontend Application	Next.js (App Router) and React	Provides routed pages, layouts, and interactive UI components for all dashboards, registries, and decision-support views.
UI Components	TypeScript React components, lucide-react icons	Implements reusable presentational components such as DataTable, PageHeader, StatCard, Badge, Modal, ConfirmDialog, and Toast.
Service / Logic Layer	TypeScript services in src/services and analytical utilities in src/utils/analytics	Centralises Supabase data access, formula computation, recommendation generation, and decision-support snapshot retrieval.

Layer	Technology	Purpose in the System
Backend / Data Platform	Supabase	Hosts authentication, PostgreSQL data, row-level security policies, SQL functions, and views.
Database	PostgreSQL	Stores all operational, reference, and analytical data, and exposes computation functions for MTTR, MTBF, availability, PMC, and snapshot refresh.
AI Assistant	Gemini API integration with TypeScript classifier, context retrieval, and safety policy	Provides a system-wide biomedical AI copilot with role-aware retrieval and structured output contracts.
Visualisation	Custom chart components (e.g., GaugeChart) and tabular layouts	Provides analytical visualisations on the analytical dashboard and on each analytics page.

2.5 Repository and Branch Overview

The implementation is organised as a single Next.js repository. Within the application source directory src/, dashboard routes are grouped under src/app/(dashboard) using Next.js route groups, while authentication routes are isolated under src/app/(auth). Reusable layout components reside in src/components/layout, including the dashboard sidebar, top bar, and dashboard layout shell, which together provide the navigation chrome for the application. UI primitives are located in src/components/ui. The TypeScript service layer is located in src/services and is organised by domain (equipment, maintenance, PM, calibration, spare parts, disposal, training, decision support, chatbot, reports). Analytical formulas, normalisation, composite scoring, replacement-index logic, and recommendation generation are located in src/utils/analytics. Domain types are centralised in src/types/database.ts, and Supabase client helpers are located in src/lib/supabase. The database schema and policies are managed through versioned SQL migrations in supabase/migrations, currently spanning migrations 00001 to 00016, supported by deterministic seed data in supabase/seed. The branch used for the present progress reporting work is progress_report.

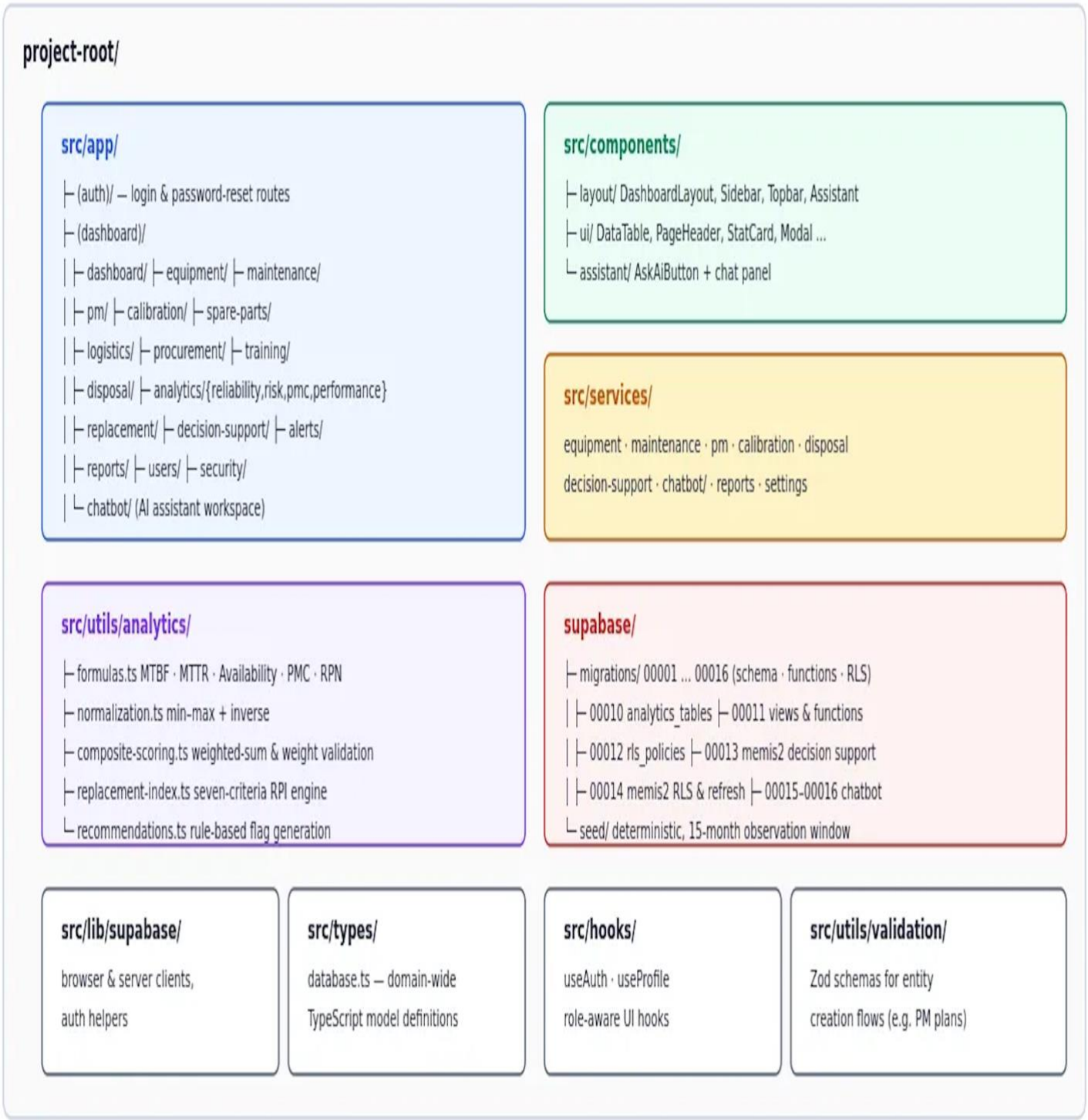

Figure 1. Repository structure of the progress_report branch

CHAPTER THREE: IMPLEMENTATION PROGRESS

3.1 System Architecture Implemented

The implemented architecture follows a clean layered structure. The user interface layer is built using Next.js App Router pages and a shared dashboard layout, with role-aware sidebar navigation and a system-wide AI assistant launcher and panel attached at the layout level. The service and logic layer, written in TypeScript, mediates all access to the database and centralises business logic. The data layer is hosted on Supabase using PostgreSQL, with explicit row-level security policies, SQL functions, materialised analytical tables, and views. The analytical and decision-support layer combines TypeScript utility functions for the proposal's equations with PostgreSQL functions and a snapshot-refresh routine that feeds the Decision Support Center. Figure 1 summarises this layered architecture as currently implemented.

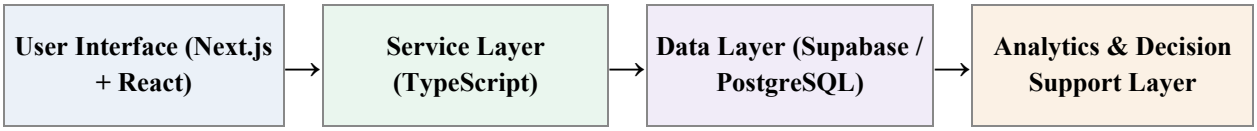

Figure 2. Overall System Architecture

3.2 User Interface and Navigation Structure

The user interface is structured around a persistent sidebar grouped into functional sections, including Dashboard, Operations, Decision Support, Reports, and Administration. Each navigation item is associated with a role list, and the sidebar dynamically filters items based on the current user's roles. The top-level analytical dashboard is exposed at /dashboard/analytical, with an additional work-order dashboard at /dashboard/work-orders. The repository defines an explicit ROUTES constant covering equipment, requests, maintenance, work orders, preventive maintenance, calibration, spare parts, logistics, procurement, training, helpdesk, alerts, disposal, decision support, reliability analytics, risk scoring, PM compliance, performance scores, replacement priority, reports, security, and the AI chatbot. Figure 2 shows the login page UI while Figure 3 shows the analytical dashboard page

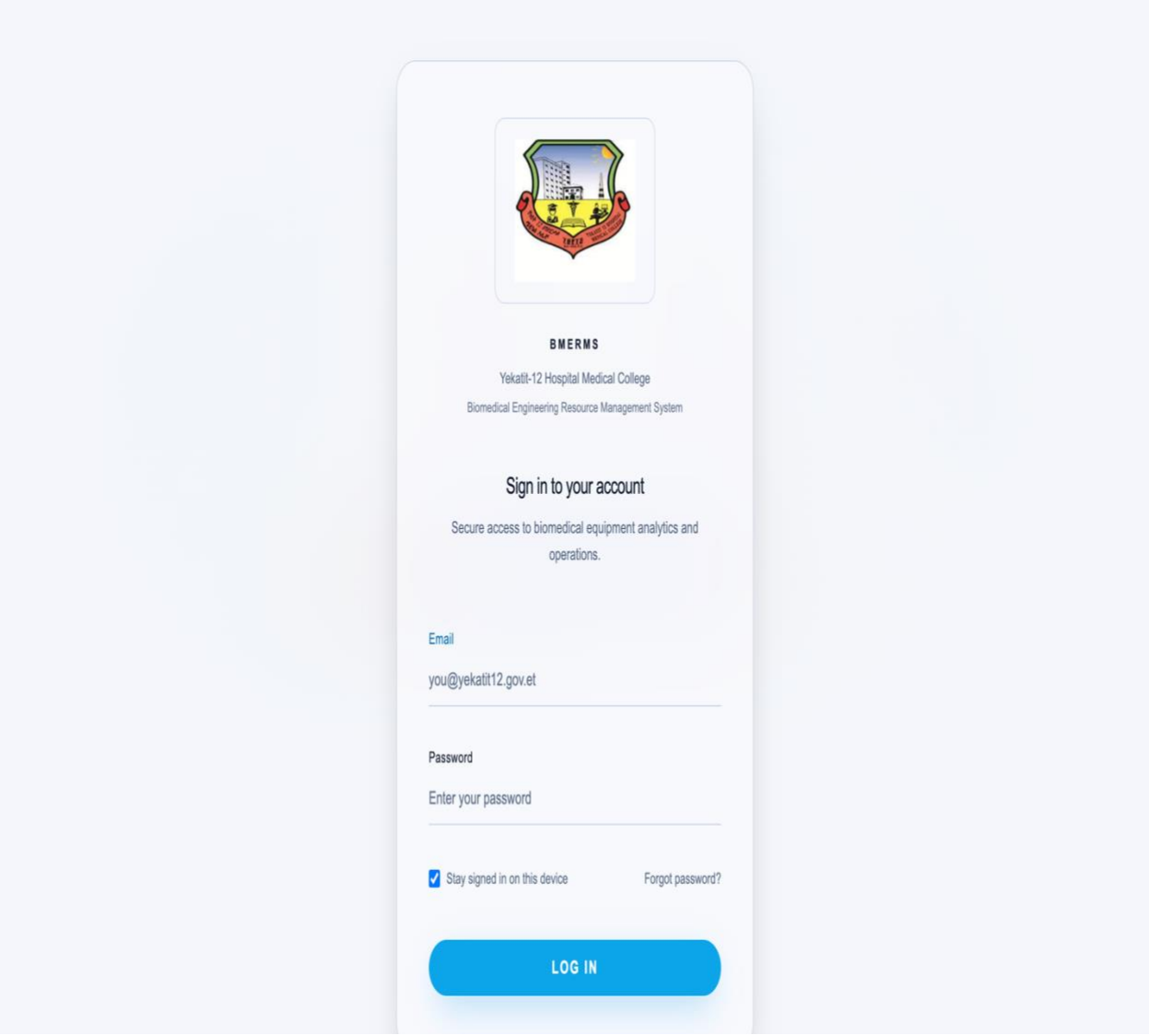

Figure 3. Login page

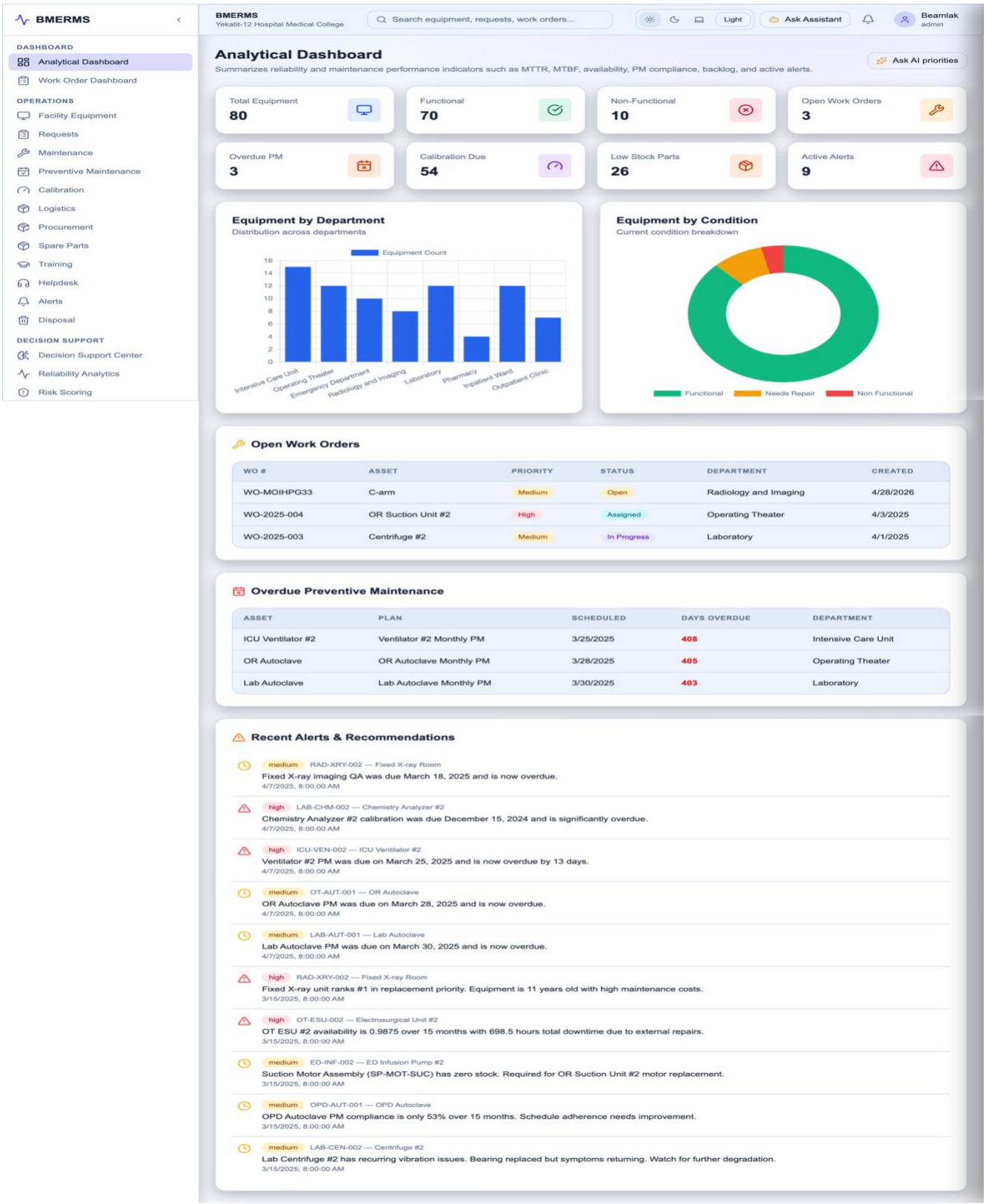

Figure 4. Analytical Dashboard

Each operational module exposes its own page or set of nested pages within src/app/(dashboard)/. For example, the maintenance area at src/app/(dashboard)/maintenance/page.tsx renders both maintenance requests and work orders in a tabbed view with structured columns for asset, department, urgency, status, and dates. The PM area at src/app/(dashboard)/pm/ exposes plans, schedules, and detail pages, including a schedule completion workflow that records duration, notes, and checklist results. The replacement area at src/app/(dashboard)/replacement/page.tsx renders the multi-criteria replacement priority ranking with explicit per-criterion scores and a final RPI badge. Table 2 summarises the implemented functional modules with selected repository evidence.

Table 2. Implemented Functional Modules and Repository Evidence

Area	Summary of Implemented Functionality
Authentication, Users & Security	Supabase authentication, password reset, sessions, roles, permissions, profiles, audit logs, and security screens.
Dashboard & Core Operations	KPI dashboards, work-order monitoring, request intake, equipment inventory, maintenance, and repair workflows.
Lifecycle Management	Preventive maintenance, calibration, spare parts, logistics, procurement, training, and disposal workflows.
Reports & Analytics	Reports for all major modules plus MTTR, MTBF, availability, RPN, PM compliance, performance scores, and replacement priority.
Decision Support	Triage queue, health scoring, clinical readiness, capacity/backlog analysis, snapshot refresh, and automated recommendations.
AI, Helpdesk & Offline Foundation	AI assistant, role-aware retrieval, helpdesk entry point, offline sync table, telemetry baseline, and QR/offline workflow foundation.

3.3 Authentication and Role-Based Access

The authentication layer uses Supabase Auth, with login and password-reset routes grouped under src/app/(auth) and a stylised authentication backdrop component. The dashboard root layout at src/app/(dashboard)/layout.tsx loads the current user using the useAuth hook, retrieves the corresponding profile via useProfile, and forwards the user's primary role and full role list into the dashboard layout, where they drive sidebar visibility through role-tagged navigation entries. The system defines five user roles: admin, technician, department_user, store_user, and viewer, each associated with a distinct subset of navigation items. The data layer enforces row-level security on

every operational, reference, and analytical table, using a SECURITY DEFINER helper function auth_user_has_role to evaluate the caller's assigned roles via the user_roles join table.

3.4 Equipment Inventory Management

The equipment inventory is implemented through the equipment_assets table, which stores a unique asset code, serial number, name, and references to category, department, manufacturer, model, vendor, and supplier, alongside installation and warranty dates, service contract expiry, condition, status, purchase information, source, notes, and an optional photo URL. Soft deletion is supported via a deleted_at column. Reference tables for departments, equipment categories with explicit criticality levels, manufacturers, models, vendors, suppliers, failure codes, maintenance action codes, calibration types, and PM templates are managed through the first reference-tables migration. The equipment registration UI integrates these reference dictionaries through cascading select inputs and validation. Figure 5 summarises the equipment lifecycle workflow as currently implemented.

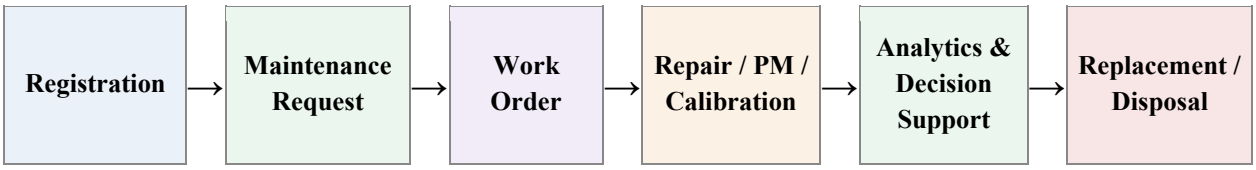

Figure 5. Equipment Lifecycle Workflow

3.5 Maintenance Request and Work-Order Management

Maintenance requests are persisted in the maintenance_requests table, with a unique request number, asset and department references, fault description, urgency, status, optional resolved-at timestamp, and notes. The associated UI provides a tabular listing with sortable columns for request number, asset, department, fault description, urgency, status, and creation date, and uses urgency and status badge components for clarity. Work orders are persisted in the work_orders table with a unique work-order number, optional link back to the originating request, asset reference, assignee, status, priority, work type, scheduled and actual start/end times, and free-text notes. The maintenance UI presents work orders in a parallel tab view with sortable columns for work-order number, asset, assignee, priority, status, work type, and start time. Status fields are constrained to controlled enumerations to support reliable analytics downstream.

3.6 Preventive Maintenance Management

Preventive maintenance is implemented through three layered tables: pm_plans, pm_schedules, and pm_completions. PM plans are created from PM templates that define checklist items and frequency in days. The PM plans creation page at src/app/(dashboard)/pm/plans/new validates input through a dedicated Zod schema and creates the underlying record through the createPMPlan service. Each schedule has a status (scheduled, in_progress, completed, overdue) and is linked back to the parent plan. The schedule detail page supports completion through a structured modal that captures the completion date, duration in hours, notes, and per-item checklist results, after which the schedule status is updated to completed. The PM seed data spans approximately two years and provides a realistic dataset for compliance computation, with overall compliance averaging approximately seventy-five percent and variation by department.

3.7 Calibration Management

Calibration is supported through a calibration_types reference table, a calibration_requests table for the operational request lifecycle, a calibration_records table for executed calibrations with date, next-due date, technician, and pass/fail/adjusted result, and a calibration_certificates table for certificate metadata. The calibration UI surfaces upcoming due dates, overdue calibrations, and result history. Recommendation flag generation in src/utils/analytics/recommendations.ts emits a calibrate_soon flag with appropriate severity when an asset's calibration becomes overdue.

3.8 Spare Parts, Logistics, and Procurement Support

Spare parts management is implemented through the spare_parts catalog, the stock_receipts table, and the stock_issues table. The spare-parts page at src/app/(dashboard)/spare-parts exposes four tabs (catalog, receipts, issues, low stock), with modal forms for adding parts, recording receipts, and recording issues. Items below the configured reorder level are automatically surfaced in the low-stock tab and are also exposed through a low-stock view at the database layer for cross-module use, including by the AI assistant. The logistics hub at src/app/(dashboard)/logistics provides MEMIS-style entry points for item receive, item request, item approval and issue, stock balance, and usage linkage to maintenance.

Procurement tracking is implemented through the procurement_requests table introduced by the MEMIS 2.0 decision-support migration. Each procurement request stores a unique request

number, title, justification, status, priority, requested-by reference, department reference, and expected delivery date. Status values are constrained to a structured taxonomy aligned with MEMIS lookup values, including requested, approved, ordered, in_transit, delivered, and canceled. RLS policies on procurement_requests permit insertion by admins, technicians, store users, and department users, while updates are restricted to admins, technicians, and store users, and deletion is restricted to admins.

3.9 Training Management

Training management is implemented through training_requests, training_sessions, staff_training_records, and equipment_training_records. The training UI supports submission of training requests, viewing of scheduled sessions, and tracking of staff certification and equipment-specific training topics. The AI assistant exposes a training_status capability that can summarise pending training requests, scheduled sessions, equipment training coverage gaps, and overdue training requests for critical assets when grounded data is available.

3.10 Disposal Management

Disposal is implemented through disposal_requests for the recommendation and approval lifecycle, and disposed_assets for the post-disposal record. The disposal UI exposes two tabs for requests and disposed assets, and includes a structured submission modal capturing asset, reason, proposed method, and notes, alongside approve and reject confirmation dialogs. The disposal service generates request numbers automatically and records approval timestamps and approver identifiers when requests are approved.

3.11 Reporting and Dashboard Features

A reports hub at src/app/(dashboard)/reports/ exposes structured report builders for equipment inventory, maintenance history, PM completion, calibration, training, spare-parts usage, and disposal. Each report is implemented through a parameterised page at src/app/(dashboard)/reports/[type]/page.tsx that selects appropriate columns, filters, and a fetch function from the reports service. The analytical dashboard exposes overall KPI cards based on the v_dashboard_stats SQL view, including total equipment, functional and non-functional counts, open work orders, overdue PM count, calibration due soon, low-stock parts, pending disposals, and active critical alerts.

3.12 AI Assistant, Alerts, and Notification Support

The platform integrates a system-wide biomedical AI copilot, exposed both as a global launcher attached to the dashboard layout and as a dedicated full workspace at the /chatbot route. The assistant is implemented in src/services/chatbot/ and is composed of an intent classifier, a Supabase-backed context retrieval service, a structured safety policy that returns explicit decisions (answer, limited_answer, check_manual, escalate, refuse), a Gemini-based generation client, and a structured-output validator. Persistence and row-level security for chat sessions, chat messages, session memory, and telemetry are defined in migrations 00015 and 00016. Contextual entry points are integrated within equipment detail, work-order detail, PM, analytics risk, decision-support, and logistics views through the AskAiButton component, allowing users to launch the assistant with a domain-relevant seed prompt.

Alerts are managed through the recommendation_flags table, which records flag type, severity, message, supporting details, acknowledgement state, and optional expiry. Flag types currently implemented include high_risk, replacement_candidate, overdue_pm, prioritize_pm, calibrate_soon, part_shortage, warranty_expiring, and contract_expiring. The alerts page at /alerts lists open flags, while the dashboard exposes the count of active critical alerts and the decision-support center incorporates open-flag counts into the equipment health score.

3.13 Offline and Technician Workflow Support

A foundation for offline technician workflow has been established. The MEMIS 2.0 decision-support migration introduces an offline_sync_events table that stores client action identifiers, actor identifiers, target entity references, action types, payloads, sync status, and synchronisation timestamps. The post-upgrade baseline seed includes a representative offline sync event for an in-progress work-order status update. RLS on offline_sync_events permits authenticated insertion and admin/technician updates. A QR-based asset access flow and a complete offline-first UI layer are planned but not yet implemented; this is documented as remaining work in Chapter Seven.

3.14 Decision-Support Layer

The analytical and decision-support layer is the principal differentiator that distinguishes the BMERMS prototype from a conventional inventory or documentation system. The layer is implemented as a hybrid of TypeScript utility functions and PostgreSQL functions and views, with a snapshot-refresh routine that materialises decision-support outputs for fast retrieval by the user interface. Figure 6 illustrates the data flow from operational records through metric computation to ranked recommendations. Table 3 summarizes the decision-support components.

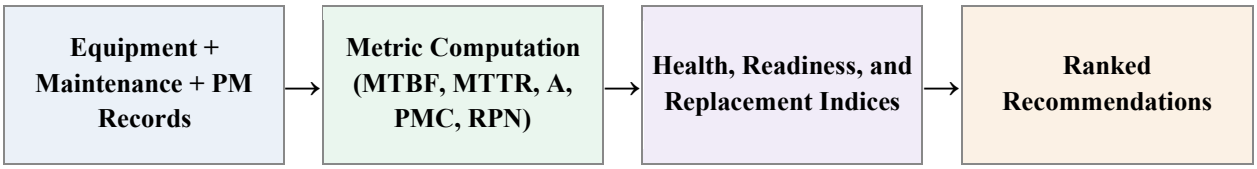

Figure 6. Data Flow

Table 3. Decision-Support and Analytics Components

Score / Index	Criteria & Weights	Formula / Aggregation	Status
Risk Priority Number (RPN)	Severity (S, 1-10), Occurrence (O, 1-10), Detectability (D, 1-10)	$RPN = S \times O \times D$ Range 1-1000; classified low<80, medium 80-199, high 200-499, critical >=500	Implemented
Availability	MTBF, MTTR	$A = MTBF \div (MTBF + MTTR)$ Ratio 0-1	Implemented
MTBF	Operational hours, failure count	$MTBF = T\_operational \div N\_failures$	Implemented
MTTR	Maintenance hours, repair count	$MTTR = T\_maintenance \div N\_repairs$	Implemented
PM Compliance (PMC)	PM completed, PM scheduled	$PMC = completed \div scheduled \times 100$	Implemented
Replacement Priority Index (RPI)	availability 0.20, age 0.15, failure_rate 0.15, maintenance_burden 0.15, risk 0.15, spare_parts 0.10, cost 0.10	$TS\_i = \Sigma(w\_j \cdot s\_ij)$ Min-max normalize each criterion (availability and spare_parts inverse-normalized), then aggregate.	Implemented (defaults from code)
Equipment Health Score	Reliability 35 (availability × 35), PM 25 (PMC/100 × 25), Risk 25 ((1 - min(0.35, RPN/1000)) × 25), Status/Flags 15 (condition penalty + open-flag penalty)	$Health = Reliability + PM + Risk + Status/Flags$ Sum of weighted components, rounded; minimum value 1	Implemented
Clinical Readiness Score	Functional and active essential equipment / total essential (high or critical criticality category)	$Readiness = functional\_active \div essential\_total \times 100$	Implemented
Triage Priority Score	Severity-weighted open-flag score (critical 45, high 25, medium 10, low 4) + RPN/15 (capped 40) + max(0, (80-PMC)/2) + max(0, 20-replacement_rank)	$Priority = flag\ score + RPN \div 15 + max(0, (80 - PMC) \div 2) + max(0,20 - rank)$ Additive aggregation, rounded	Implemented

Score / Index	Criteria & Weights	Formula / Aggregation	Status
Workload / Capacity Score	Open assignments, overdue assignments, estimated_hours, capacity_hours (default 8 h/day)	$backlog\_delta = \Sigma(estimated\_hours) - capacity\_hours$	Implemented
Composite Performance Score	Generic weighted sum of normalized criteria	$TS\_i = \Sigma(w\_j \cdot s\_ij)$ Weights validated to sum approximately 1.0	Foundation implemented; criterion-set finalization required

3.15 Summary of Analytical Strengths and Remaining Gaps

The analytical layer demonstrates substantive alignment with the proposal. Equations 1 to 7 are explicitly implemented, decision-support outputs are materialised through a structured snapshot refresh, and explainable health scores and readiness scores are exposed in the user interface. The replacement priority module operationalises the AHP-aligned weighted-sum approach described in the proposal's methodology section. The remaining analytical gaps include the integration of all analytical outputs into a single action-oriented **"Engineer Command Center"** interface, the connection of every analytical view to fully live recomputation triggers (rather than partly snapshot-driven seeded data), and the formal computational verification step in which a sample of system outputs is compared against manually computed reference values.

3.16 DATABASE, DATA MODEL, AND SECURITY

The system uses a Supabase-hosted PostgreSQL database as its primary data layer. The database structure is managed through versioned SQL migration files, which define the required tables, relationships, constraints, indexes, triggers, views, functions, row-level security policies, and seed data. The migration sequence covers the full system scope, including authentication and profiles, equipment asset management, maintenance, preventive maintenance, calibration, spare parts and logistics, training, disposal, analytics, decision-support snapshots, chatbot persistence, telemetry, and evaluation extensions. This provides a controlled and reproducible database structure that can be maintained consistently across development, testing, and deployment environments.

The data model is organized around core biomedical equipment lifecycle operations. Major operational entities include departments, equipment categories, manufacturers, equipment models, vendors, suppliers, equipment assets, equipment locations, maintenance requests, work orders, maintenance events, downtime logs, preventive maintenance plans and schedules, calibration requests and records, spare parts, stock receipts and issues, training records, disposal requests, and disposed assets. The equipment asset table acts as one of the central entities, linking inventory information with maintenance history, preventive maintenance, calibration, spare-parts usage, training, disposal, and analytical evaluation. Reference and lookup tables are used to standardize repeated values such as departments, equipment categories, failure codes, maintenance actions, calibration types, PM templates, FMEA risk scales, scoring weights, and MEMIS-aligned lookup values. This reduces duplication and improves consistency across the system.

The database also supports analytical and decision-support functions. It includes tables for reliability metrics, risk scores, PM compliance metrics, equipment performance scores, replacement priority scores, recommendation flags, equipment health snapshots, clinical readiness snapshots, triage queues, repeat repair flags, escalation events, workload-capacity snapshots, and offline sync events. SQL functions are used to compute key indicators such as MTTR, MTBF, availability, PM compliance, and decision-support snapshots. Dashboard and reporting views summarize operational data for easier presentation in the user interface. These structures allow the system to move beyond simple record keeping and provide performance evaluation, prioritization, and decision-support outputs.

Security is implemented through Supabase Row-Level Security on application tables. Access control is based on authenticated users, profiles, roles, and user-role assignments. A helper function checks the role of the current user before allowing sensitive operations. In general, authenticated users may view relevant records, while create, update, and delete permissions are restricted based on roles such as administrator, technician, store user, and department user. Reference table changes are limited to administrators, operational updates are mainly limited to administrators and technicians, and decision-support tables are readable by authenticated users but writable only by authorized roles. Chatbot-related tables also apply ownership-based access, with administrative override where necessary.

Data quality is maintained through database constraints, generated columns, validation rules, and audit logging. Examples include unique active asset codes, non-empty fault descriptions, standardized work-order priority values, controlled criticality levels, generated RPN and risk-level values, foreign-key enforcement, and timestamp triggers. Application-level validation is also supported through validation schemas before data is inserted into the database. The audit log records important actions, affected entities, old and new values, user context, IP address, and timestamp. Seed data is currently used to demonstrate the full system workflow, but it is designed to be replaced by real hospital data through structured mapping from paper records, Excel files, and other existing hospital records into the appropriate system tables. During real data replacement, sensitive identifiers should be anonymized and foreign-key relationships should be validated before import.

CHAPTER FOUR: ALIGNMENT WITH PROPOSAL OBJECTIVES

4.1 Objective-by-Objective Progress Review

This section maps the implementation status against each of the four specific objectives defined in the original proposal. Table 4 provides a structured summary, while the surrounding paragraphs offer a brief discursive review.

Table 4. Alignment of Implementation with Proposal Objectives

Proposal Objective	Implemented Evidence	Status	Remaining Work
Objective 1: Design a structured framework for integrating equipment inventory, maintenance history, and operational data.	Comprehensive relational schema with equipment_assets, maintenance_requests, work_orders, maintenance_events, pm_plans, pm_schedules, pm_completions, calibration_records, spare_parts, stock_movements, training, and disposal tables; service-layer abstractions in src/services; consistent validation in src/utils/validation/.	Achieved	Migration of seeded records to real data and consolidation of the unified Engineer Command Center.
Objective 2: Develop analytical models that quantify equipment performance, reliability, and maintenance effectiveness using indicators such as MTTR, MTBF, availability, and preventive maintenance compliance.	TypeScript implementations of computeMTTR, computeMTBF, computeAvailability, and computePMC in src/utils/analytics/formulas.ts; SQL functions fn_compute_mttr, fn_compute_mtbf, fn_compute_availability, and fn_compute_pmc in migration 00011; persistence in equipment_reliability_metrics and pm_compliance_metrics; analytics pages exposing the computed values.	Achieved	Computational verification step against manual reference computations and automatic recomputation triggers.
Objective 3: Implement risk-based and multi-	FMEA RPN through computeRPN and the equipment_risk_scores	Achieved	Sensitivity analysis across

Proposal Objective	Implemented Evidence	Status	Remaining Work
criteria decision models for prioritising equipment maintenance and replacement.	generated column; min–max normalisation in src/utils/analytics/normalization.ts; weighted-sum scoring in src/utils/analytics/composite-scoring.ts; replacement priority engine in src/utils/analytics/replacement-index.ts producing replacement_priority_scores; explainable health and readiness through refresh_decision_support_snapshots.		alternative weight profiles and visualisation of stability of top-ranked priorities.
Objective 4: Develop and evaluate a prototype decision-support system that generates equipment performance indicators, risk rankings, and replacement prioritisation outputs using hospital equipment data.	End-to-end working prototype with operational modules, analytics pages, Decision Support Center, recommendation flags, AI assistant, and alerts; structured workflows for triage and replacement; reports across the full lifecycle.	Substantially achieved (prototype level)	Formal evaluation with biomedical engineers, including usability testing, satisfaction survey, and computational verification against manual outputs.

4.2 Implemented Features Compared with Proposed Scope

The proposal scoped the system to equipment managed by, or relevant to, biomedical engineering departments across major clinical areas including intensive care, operating theatres, emergency services, imaging, inpatient wards, and laboratories. The current implementation supports each of these functional areas through structured department records, criticality-aware equipment categorisation, and category-specific PM templates. The proposal further committed the system to generate a high-risk equipment list, low-availability and high-maintenance-burden lists, preventive

maintenance compliance reports and overdue task lists, and replacement priority rankings; each of these outputs is now produced by the implemented analytics and decision-support layers.

4.3 Evidence of Technical and Academic Progress

Several lines of evidence support the conclusion that meaningful technical and academic progress has been achieved. First, the analytical equations stipulated in the proposal are explicitly implemented and traceable to identified files in the repository. Second, the database schema includes auditability and data-quality features beyond minimum requirements, including audit logging, soft deletion, generated columns for derived values, and structured CHECK constraints. Third, the platform has progressed beyond core inventory and maintenance and incorporates a unified Decision Support Center that materialises explainable health scores, readiness, triage, and capacity simultaneously. Fourth, the system-wide AI assistant integrates structured intent classification, role-aware retrieval, and a safety policy that returns explicit decisions, supporting examiner-relevant claims about safe AI deployment in clinical engineering contexts.

4.4 Contribution Beyond Basic Documentation Systems

The proposal explicitly motivated this thesis by pointing out that existing platforms such as MEMIS and similar hospital-level systems improve equipment information visibility but lack analytical decision-support capability. The current implementation directly addresses this gap. The Decision Support Center, the replacement priority module, the recommendation-flag generator, and the explainable health and readiness scores collectively translate routine equipment data into ranked engineering priorities. As a result, the implemented prototype is materially more than a record-keeping system: it is a hospital-level decision-support platform that can support biomedical engineering teams in directing limited resources toward equipment with the greatest operational and clinical impact.

CHAPTER FIVE: REMAINING WORK

5.1 Current Testing and Verification Status

The current testing status combines automated and manual elements. Automated tests exist for the AI copilot pipeline under src/services/chatbot/__tests__, including assistant intro, response pipeline, request validation, copilot core, Gemini provider, and structured-context fallback tests, runnable via npm run test:chatbot. Outside the copilot, manual smoke testing has been performed against the seeded demonstration dataset; comprehensive role-specific end-to-end testing remains required.

5.2 Functional Limitations

The current implementation includes role-based access structures, but role-specific end-to-end testing remains required before final deployment. Some workflow transitions, such as procurement approval routing, training competency reporting, and disposal approval chains, are foundationally implemented but not yet stress-tested with parallel users. Notification and alert delivery beyond in-application surfaces (for example email or SMS) is not in scope of the current prototype.

5.3 Analytical Limitations

Some analytical views depend on seeded or snapshot data; therefore, additional work is required to ensure automatic recomputation when new operational records are entered. The Equipment Health Score, Clinical Readiness Score, Triage Priority Score, and Workload Capacity values are produced by the refresh_decision_support_snapshots function, which currently must be invoked to refresh same-day snapshots. The Replacement Priority Index uses the default weights documented in Table 3; weight calibration with hospital biomedical engineers remains required before final deployment.

5.4 Data Collection and Evaluation Limitations

The current version remains a prototype and requires validation using real hospital equipment inventory and maintenance records. The seeded demonstration dataset is sufficient to exercise the analytical pipeline and dashboards but is not a substitute for evaluation against operational data from a selected hospital setting. Biomedical engineer evaluation, including expert review of triage queue ordering and replacement recommendations, is required before claims of operational fitness can be made.

5.5 UI/UX and Deployment Limitations

UI/UX limitations include the absence of comprehensive accessibility audits, limited responsive testing on representative low-end devices, and a settings surface that is functional but not yet localized. Deployment limitations include the absence of a hardened production environment for hospital use; production deployment, observability, and backup procedures are required prior to a hospital-level pilot.

5.6 Remaining Work Before Final Defense

Table 5 summarizes the remaining work before final thesis defense.

Table 5. Current Limitations and Planned Corrective Actions

Area	Current Limitation	Effect on Final System	Planned Action
Real hospital data	System currently demonstrated using seeded fifteen-month dataset.	Limits external validity of analytical outputs.	Collect, clean, and migrate real data like inventory, maintenance, PM, calibration, and disposal records, with anonymisation and field mapping.
Analytical recomputation	Some analytical tables seeded; refresh function manual.	Outputs may not reflect every recent operational event automatically.	Add database triggers for automatic recomputation on relevant insert and update events; expose a scheduled refresh option.
Unified Decision Interface	Decision-support outputs distributed across multiple pages.	Engineers must navigate between modules to combine insights.	Consolidate triage, health, readiness, workload, replacement, and recommendation views into a single action-oriented Engineer Command Center.
Sensitivity analysis	Multiple weight profiles supported in data model only.	Robustness of weighting choices not	Expose UI control to switch weighting profiles, run RPI under

Area	Current Limitation	Effect on Final System	Planned Action
		yet visible to examiner.	alternative weights, and display top-priority stability.
Computational verification	Manual verification step against published equations not yet documented.	Required by the proposal's evaluation strategy.	Select representative devices, hand-compute Equations 1 to 7, compare with system output, and document outcomes in the final thesis.
Offline / QR workflow	Database scaffolding present; complete end-to-end UI not implemented.	Limits adoption in low-connectivity contexts.	Implement offline-first technician page, local queue, sync resolution, and QR-based asset access.
Evaluation with BMEs	Usability testing and satisfaction survey not yet performed.	Required by the proposal's evaluation strategy.	Conduct structured usability testing and satisfaction survey with biomedical engineers.
Role / RLS validation	RLS policies present; role-specific user journeys not yet end-to-end verified.	Some scenarios may surface unintended access patterns.	Run scenario-based end-to-end tests for each role and document the verification outcomes.
Documentation	Implementation captured in this progress report; final thesis chapters pending.	Final thesis report not yet complete.	Complete methodology, results, limitations, and screenshots chapters of the final thesis.

CHAPTER SIX: CONCLUSION

6.1 Summary of Progress

The progress report documents an integrated, repository-grounded implementation of the BMERMS platform on the progress_report branch. All operational modules from the original proposal — inventory, maintenance and work orders, preventive maintenance, calibration, spare parts and logistics, procurement, training, and disposal — are implemented, together with reporting and an analytical layer covering reliability (MTBF, MTTR, Availability), risk (RPN), PM compliance (PMC), composite performance scoring, equipment health, clinical readiness, triage prioritization, workload capacity, and a multi-criteria Replacement Priority Index with explicitly documented default weights.

6.2 Academic and Practical Value

Academically, the implementation provides a transparent traceability between thesis proposal equations and source-code symbols, demonstrating the application of FMEA, reliability theory, PM compliance, and multi-criteria weighted-sum decision-making to the hospital equipment management domain. Practically, BMERMS offers biomedical engineering departments in Ethiopian hospitals a single, role-aware platform that can prioritize action, surface replacement candidates, and consolidate the operational records needed for evidence-based equipment management in resource-constrained healthcare facilities.

6.3 Readiness for Continued Development and Final Evaluation

The prototype is ready for the next development phase, which centers on real hospital data migration, automated analytics refresh, weight calibration, role-specific end-to-end testing, AI copilot safety evaluation, codebase generalization, and a hospital-level pilot in a selected Ethiopian hospital setting. With these steps in place, the system is positioned for final examiner evaluation and for deployment as a hospital-level prototype supporting biomedical engineering departments in Ethiopian hospitals.

BIBLIOGRAPHY

[1] A. E. Woldeyohanins, N. M. Molla, A. W. Mekonen, and A. Wondimu, "The availability and functionality of medical equipment and the barriers to their use at comprehensive specialized hospitals in the Amhara region, Ethiopia," Frontiers in Health Services, vol. 4, art. 1470234, Jan. 2025, doi: 10.3389/frhs.2024.1470234.

[2] N. Sewagegn, T. Tilahun, G. Dessie, and B. Bezabih, "Assessment of the operational status of medical equipment in public hospitals in the Amhara region, Ethiopia: A sub-national study," Discover Health Systems, vol. 4, art. 131, 2025, doi: 10.1007/s44250-025-00312-9.

[3] S. H. Kabeta, T. K. Chala, and F. Tafese, "Medical equipment management in general hospitals: Experience of Tulu Bolo General Hospital, South West Shoa Zone, Central Ethiopia," Medical Devices: Evidence and Research, vol. 16, pp. 57–70, 2023, doi: 10.2147/MDER.S398933.

[4] Federal Ministry of Health Ethiopia, Ethiopian Hospital Services Transformation Guidelines: Medical Equipment Management. Addis Ababa, Ethiopia, 2018.

[5] B. Mang, Y. Oh, C. Bonilla, and J. Orth, "A medical equipment lifecycle framework to improve healthcare policy and sustainability," Challenges, vol. 14, no. 2, art. 21, 2023, doi: 10.3390/challe14020021.

[6] J. Moufid, R. Koulali, K. Moussaid, and N. Abghour, "Toward predictive maintenance of biomedical equipment in Moroccan public hospitals: A data-driven structuring approach," Applied Sciences, vol. 15, no. 20, art. 10983, 2025, doi: 10.3390/app152010983.

[7] A. H. Zamzam, A. K. A. Wahab, M. M. Azizan, and S. C. Satapathy, "A systematic review of medical equipment reliability assessment in improving the quality of healthcare services," Frontiers in Public Health, vol. 9, art. 753951, 2021, doi: 10.3389/fpubh.2021.753951.

[8] M. Hillebrecht, C. Schmidt, B. P. Saptoka, J. Riha, M. Nachtnebel, and T. Bärnighausen, "Maintenance versus replacement of medical equipment: A cost-minimization analysis among district hospitals in Nepal," BMC Health Services Research, vol. 22, art. 1023, 2022, doi: 10.1186/s12913-022-08392-6.

[9] S. Vala, P. Chemweno, L. Pintelon, and P. Muchiri, "A risk-based maintenance approach for critical care medical devices: A case study application for a large hospital in a developing country," International Journal of System Assurance Engineering and Management, accepted manuscript (repository version), 2018.

[10] L. Huang, W. Lv, Q. Huang, H. Zhang, et al., "Transforming medical equipment management in digital public health: A decision-making model for medical equipment replacement," Frontiers in Medicine, vol. 10, art. 1239795, 2024, doi: 10.3389/fmed.2023.1239795.

[11] P. Zhang, H. Zhu, Y. Qian, M. Chen, and L. Liu, "Effectiveness of failure mode and effects analysis in managing skin complications in ICU psychiatric patients: A real-world study," Frontiers in Psychiatry, vol. 16, art. 1571858, 2025, doi: 10.3389/fpsyt.2025.1571858.

[12] M. D. Al-Tahata and F. Al-Rifa'e, "Applying a data analytics approach to medical equipment maintenance management to improve the mean time between failure and availability," Jordan Journal of Mechanical and Industrial Engineering, vol. 17, no. 1, pp. 33–40, Mar. 2023.

[13] H. Almomani and A. H. Aldaihani, "Using computerized maintenance management system (CMMS) in roads maintenance operations," International Journal of Engineering Systems, 2021.

[14] OECD, Handbook on Constructing Composite Indicators: Methodology and User Guide. Paris, France: OECD Publishing, 2008.

[15] SIGMA, Life-Cycle Costing, Public Procurement Brief 34. Paris, France, 2016.

[16] M. Faisal and A. Sharawi, "Prioritize medical equipment replacement using analytical hierarchy process," IOSR Journal of Electrical and Electronics Engineering, vol. 10, no. 3, ver. II, pp. 55–63, May–Jun. 2015, doi: 10.9790/1676-10325563.

[17] Federal Ministry of Health Ethiopia, Medical Equipment Management Information System (MEMIS) User Guide, Pharmaceutical and Medical Equipment Directorate, Addis Ababa, Ethiopia, 2020.

[18] A. Assefa, E. Mengesha, L. Fufa, N. Abebe, T. Abebaw, and Y. Gezahegn, Web-Based Medical Equipment Management System for Hospitals, BSc Thesis, Center of Biomedical